KEY-VALUE STORES: REDIS

MİRAC MERT GÜLER
ANTHONY KERR
MİLİCA TOPİC

AGENDA

Databases & NoSQL Overview

Database types, SQL vs NoSQL fundamentals

Key-Value Stores

Concept, operations, and performance

Redis Fundamentals

Architecture, persistence, features

Redis Data Structures

Strings, Lists, Hashes, Sets, Sorted Sets

Real-World Use Cases

Caching, sessions, leaderboards

Live Demo & Tutorial

Hands-on Redis commands & Docker

Database Overview

Understanding Database Types

Most people are familiar with SQL databases

- SQL databases use structured tables, schemas, and relationships between data.
- NoSQL databases provide flexible data models designed for scalability and high performance.

- A common type of NoSQL database is the **key-value store**, where data is stored as a key and its associated value.

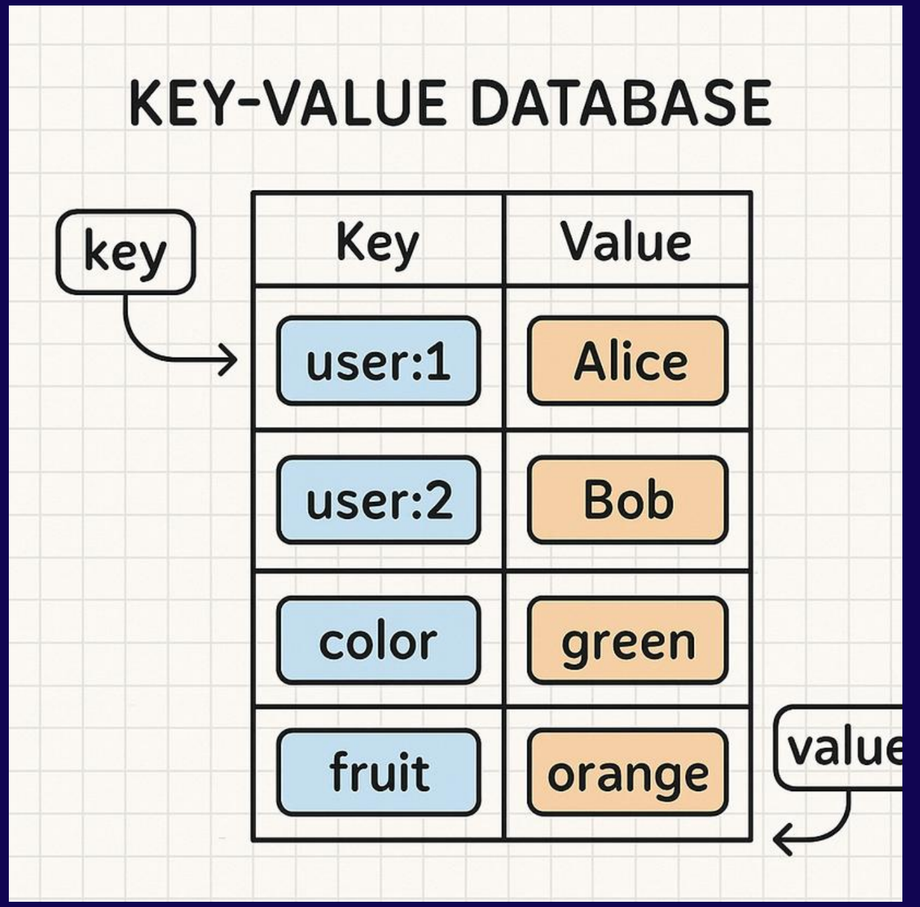
- Popular key-value stores include Redis, Amazon DynamoDB, Riak, Aerospike, and Memcached.

- used for caching, session management, real-time applications, and fast data retrieval.

WHAT IS A KEY-VALUE STORE?

A key-value store is a NoSQL database that stores information as a unique key and a value. The key is used to quickly find the associated data.

- Stores data as key-value pairs
- Each key is unique
- The value contains the actual data
- Fast and efficient for data retrieval



Key = Identifier

Unique string used to locate and access stored data.

Value = Data

The actual content stored and retrieved using its key.

BASIC KEY-VALUE OPERATIONS

SET

Create or update data with a key-value pair.

SET user:42
"Sam"

Result: Stores data (overwrires if exists)

GET

Retrieve the value associated with a specific key.

GET user:42

Returns: “Sam” (Returns Nil if empty)

DEL

Remove a key and its associated value from storage.

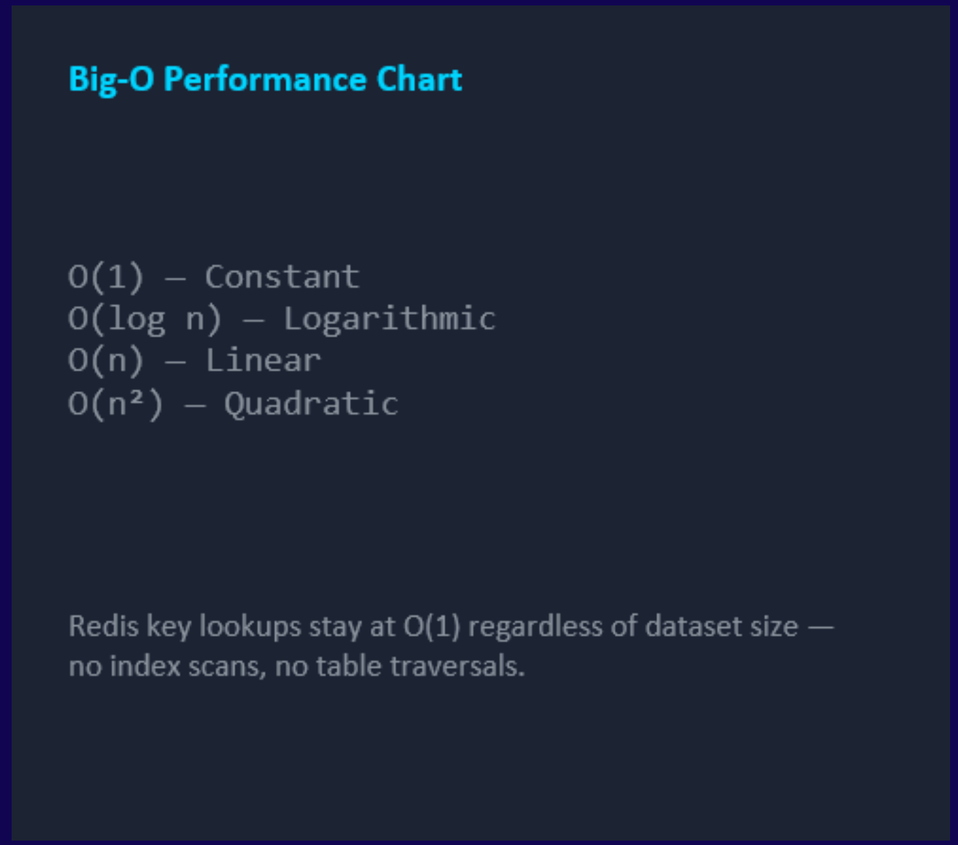
DEL user:42

Returns 1 (0 if not found)


redis-cli — root@localhost
▸ REDIS CLI v7.2.0
root@kv-store:~$ redis-cli
Connecting to 127.0.0.1:6379...
OK Connected.
— SET ——————————————————
127.0.0.1:6379> SET username "mirac"
+OK
— GET ——————————————————
127.0.0.1:6379> GET username
"mirac"
— DEL ——————————————————
127.0.0.1:6379> DEL username
(integer) 1
127.0.0.1:6379> GET username
(nil)
root@kv-store:~$

WHY ARE KEY-VALUE-STORES FAST

- Data is accessed directly using a key
- No need to search entire tables
- Simple structure reduces processing time
- Fast read and write performance
- Data is stored in RAM instead of disk



INTRODUCING REDIS

WHERE REDIS FITS

Relational Databases

PostgreSQL, MySQL

Document Databases

MongoDB

Graph Databases

Neo4j

Key-Value Databases

Redis – Fast key-based lookups

Key-Value Store

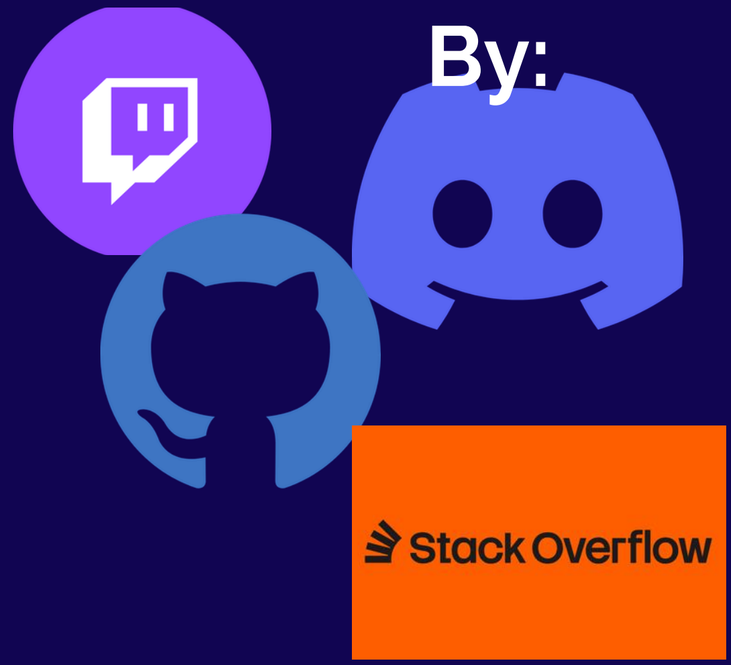
Stores data as key → value pairs

What is Redis?

Redis stands for Remote Dictionary Server. It is the most popular open-source, in-memory key-value store in the world.

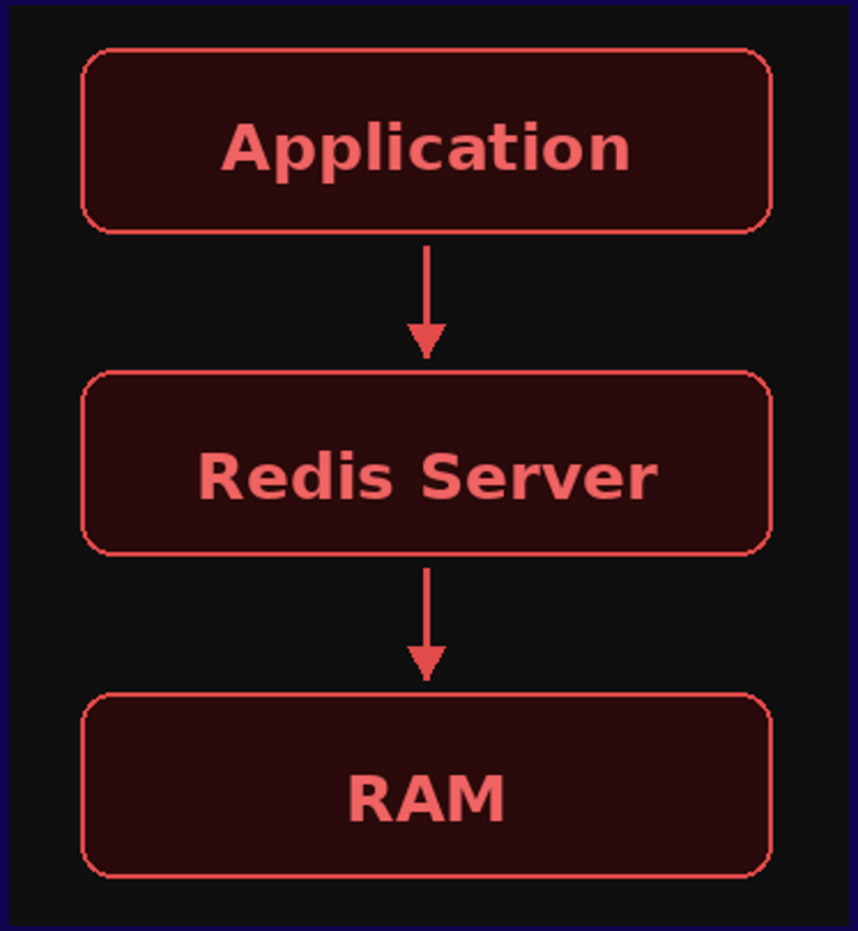
- Keeps all data in RAM for sub-millisecond read and write speeds.
- Functions primarily as an advanced key-value store.
- Supports complex values like Hashes, Lists, Sets, and JSON.
- Runs in-memory but periodically saves data to the physical disk.
- Acts as a database, a temporary cache, and a message broker.
- Prevents data corruption by running commands safely in a single thread.

Its Used By:



A message broker helps different applications communicate by sending and receiving messages between them

Redis Architecture



Applications send requests to the Redis server.

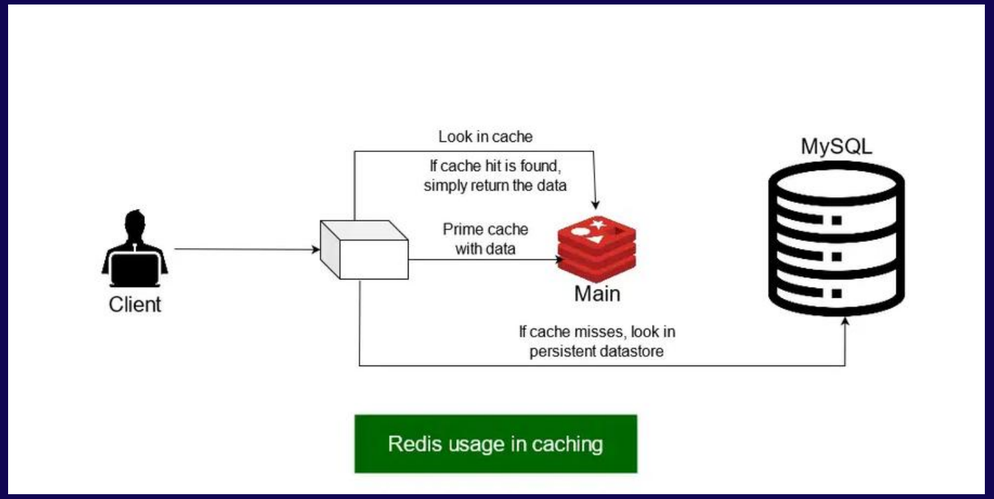
The Redis server stores and manages data in RAM

data is stored in memory, responses can be returned almost instantly

REDIS ARCHITECTURE

in Practice

- Step 1: Request Sent – The Client (user/app) requests specific data from the server.
- Step 2: Check Redis First – The system searches the ultra-fast Redis cache for the requested data.
- Step 3: Cache Hit (Fast Path) – If the data is found in Redis, it is returned to the user instantly.
- Step 4: Cache Miss (Slow Path) – If the data is missing from Redis, the system looks it up in the slower MySQL database.
- Step 5: Prime the Cache – The system delivers the MySQL data to the user and saves a copy in Redis for future requests.



WHAT HAPPENS IF THE SERVER SHUTS DOWN?

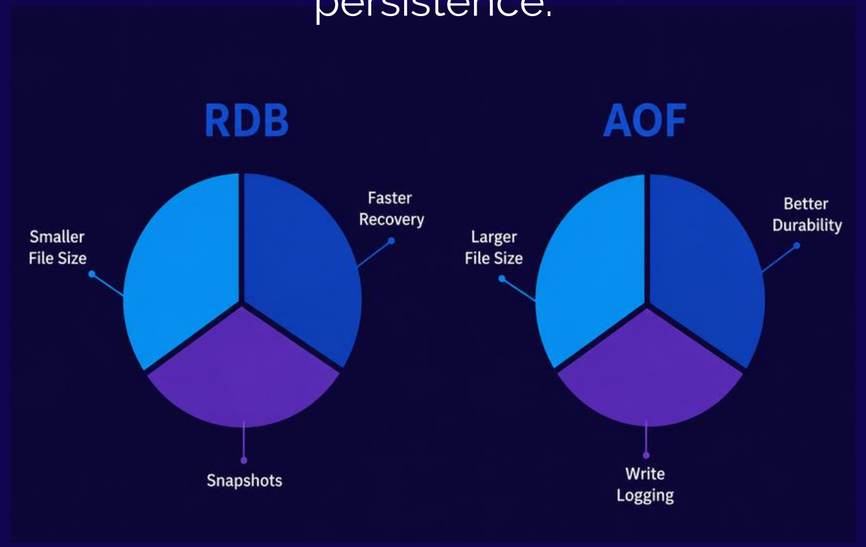
To solve this problem, Redis supports persistence.

first method

redis database

RDB

Snapshot backup at intervals



second method

append only file

AOF

Transaction log for durability

RDB 📸
Takes snapshots
change
Faster
Smaller files
May lose recent data

AOF 📝
Records every

Safer
Larger files
Almost no data loss

In Memory Storage

- Redis stores data in RAM.
- RAM is much faster than disk storage.
- This allows extremely low latency.

- **NEGATIVES:**
- RAM is more expensive.
- Data loss risk without persistence.

RAM → nanoseconds

Disk → milliseconds

For this reason, Redis combines in-memory speed with persistence options to provide both performance and reliability.

O(1)

Constant Time Complexity

Many Redis operations run in O(1) time (constant time).

- Data can be accessed directly using its key.
- Lookup speed stays almost the same regardless of data size.
- 10 records → Fast
- 10 million records → Still fast
- Provides predictable performance as applications grow.
- One reason Redis is widely used in large-scale systems.

Used when Redis can directly find a value using its key.

O(log N)

- Some Redis operations use O(log N) complexity.
- Used by advanced data structures like Sorted Sets.
- Redis uses optimized indexing to keep operations fast.
- Performance remains strong even with large datasets.
- Ideal for leaderboards, rankings, and sorted data.

Used when Redis must keep data sorted or ranked.

O(log N) = Slightly slower than O(1), but still very efficient

REDIS DATA STRUCTURES

SUPPORTED DATA STRUCTURES

String

Single value storage for text, numbers, or binary data.

usernam
e

List

Ordered collection of strings, supports push/pop operations.

task
queue

Hash

Object-like structure with field-value pairs.

user
profile

Set

Unordered collection of unique elements.

unique users

Sorted Set

Ranked collection with scores for ordering elements.

leaderboar
d

Stream

Append-only log for event streaming and messaging.

chat
messages

REDIS USE CASES

REAL-WORLD USE CASES

Caching

Store frequently accessed data in memory to reduce database load.

Session Management

Manage user sessions with automatic expiration for web applications.

Real-time Analytics

Process and aggregate live data streams for instant insights.

Message Queues

Enable asynchronous communication between distributed services.

Leaderboards

Build ranked lists using sorted sets for gaming and competitions.

Rate Limiting

Control API request frequency to prevent abuse and overload.

→Stores user sessions and temporary state

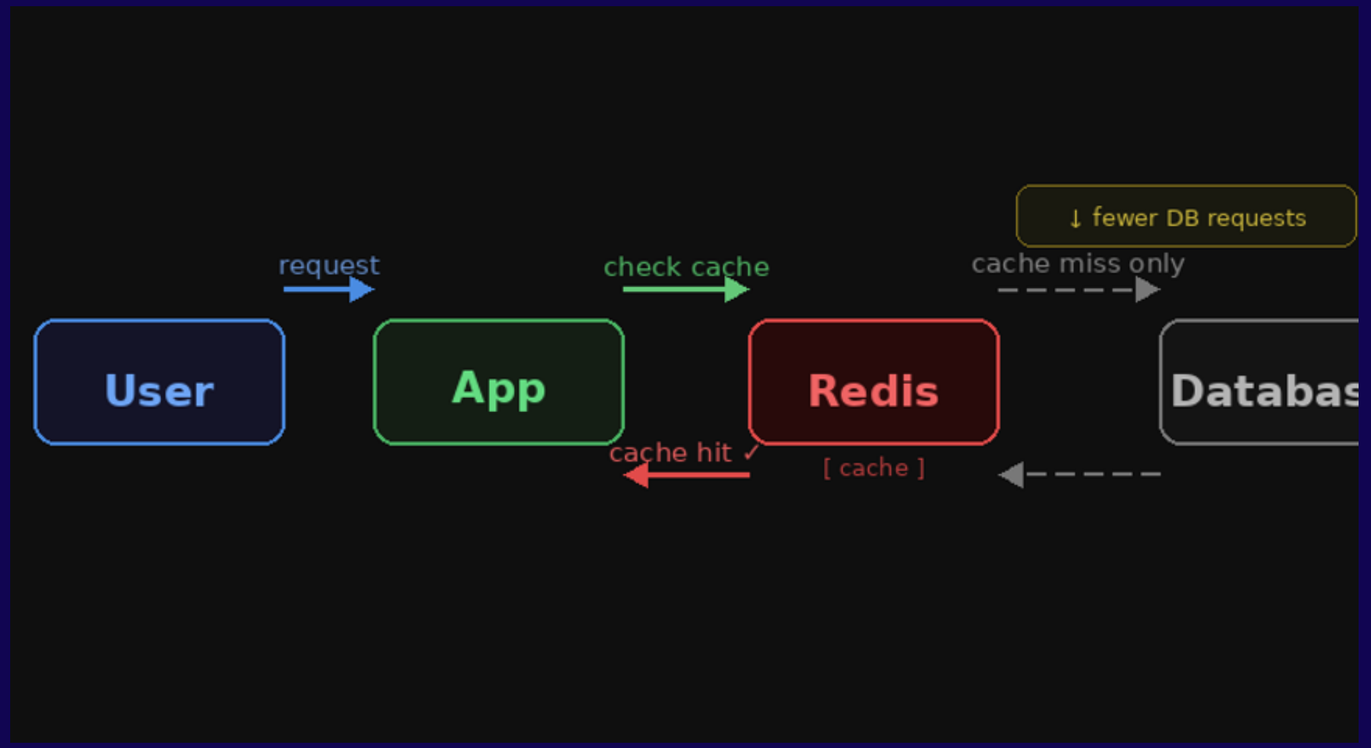
→ Caches frequently requested content

→ Uses Redis caching to reduce database load

CACHING

Redis as a Cache:

Frequently accessed data stored in Redis
Reduces database load
Improves response times



Session Management

Session Management

Web applications store:

login sessions

tokens

temporary user data

Redis is ideal because:

fast access

expiration support

Leaderboard

s

Gaming Leaderboards

Redis Sorted Sets provide:

automatic ranking

fast score updates

fast top-N queries

Perfect for:

- multiplayer games
- competitions

REDIS VS RELATIONAL DATABASES

REDIS	RELATIONAL DB
In-memory	Disk-based
Very fast	Slower
Simple queries	Complex queries
No joins	Supports joins
Best for speed	Best for structure

Advantages of Redis

Extremely fast
Easy to scale
Simple API
Flexible data structures
Excellent for real-time systems

Disadvantages of Redis

Disadvantages of Redis

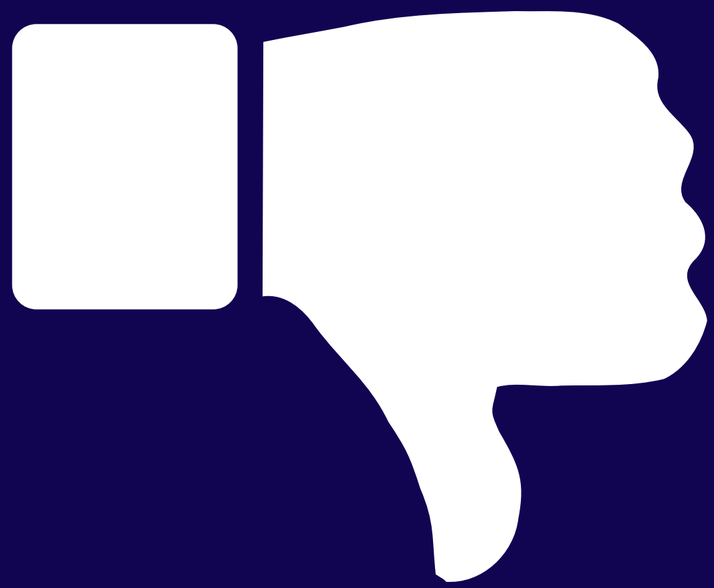
RAM cost

Limited complex querying

Not ideal for relational data

Persistence complexity

Memory limitations



When NOT To Use Redis?

- When complex SQL joins are required
- When data must be stored permanently with minimal risk of loss
- When datasets are much larger than available RAM
- When advanced relational queries are needed

Redis is often used together with relational databases rather than replacing them completely.

DEMO SCENARIO

DEMO FLOW

Start Redis

Launch container and open redis-cli interface.

Add Scores

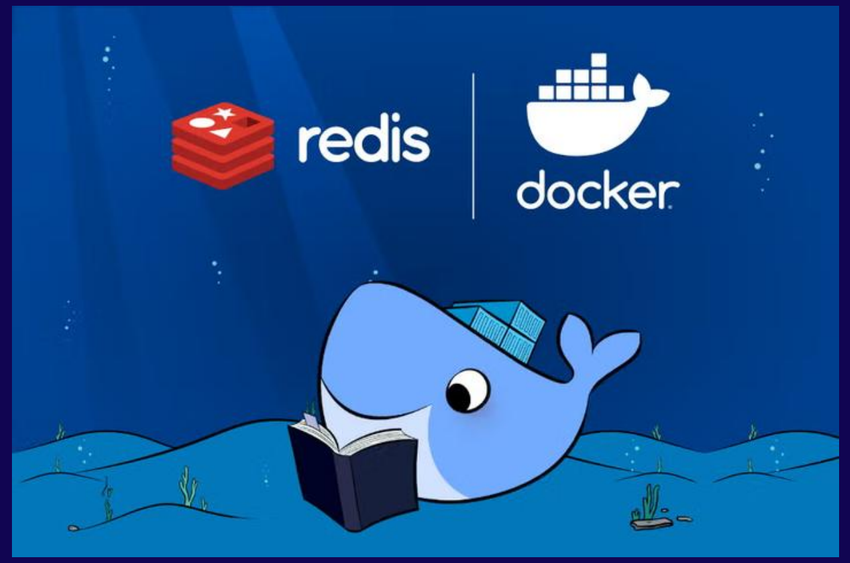
Use ZADD to add player scores and retrieve rankings.

Test Expiration

Demonstrate key expiration with SET EX and TTL.

IN THE DEMO WE WILL:

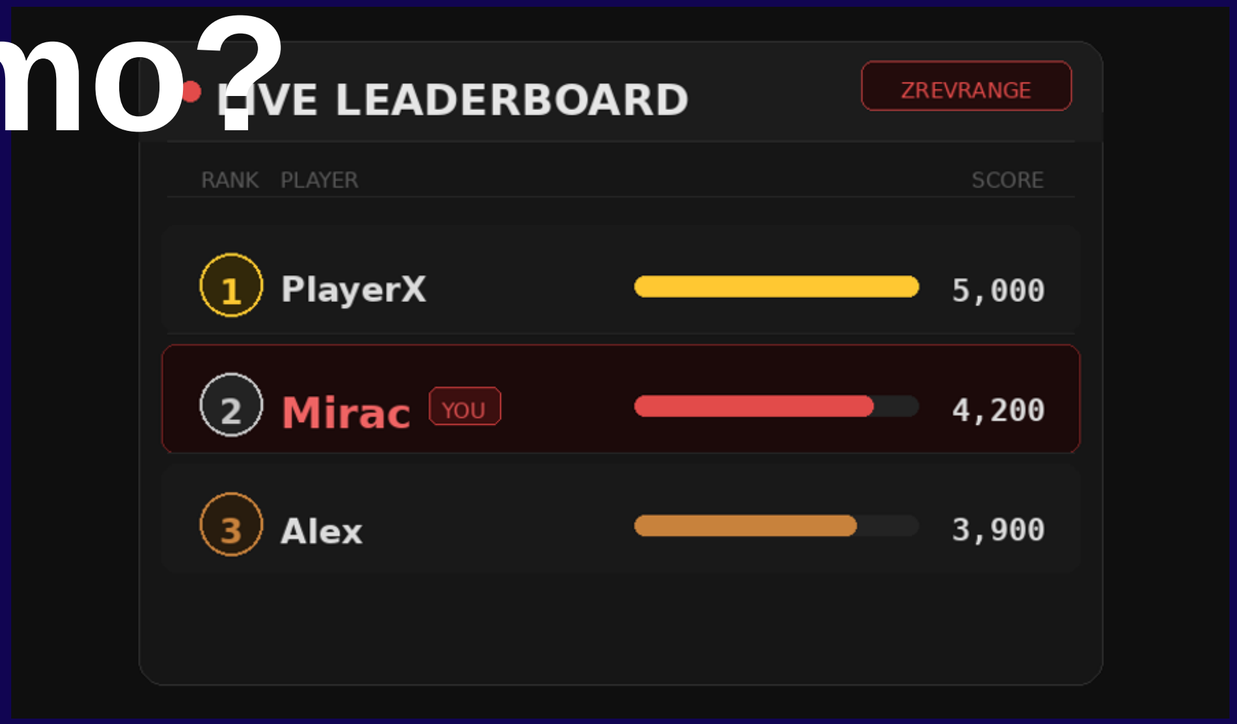
- Start Redis Container
- Connect to redis-cli
- Create Leaderboard
- Display Rankings
- Update Rankings
- Demonstrate Expiration



Why This Demo?

Why This Scenario?

- Requirements:
- very fast updates
- real-time ranking
- frequent reads/writes
- Redis performs extremely well for this type of workload.

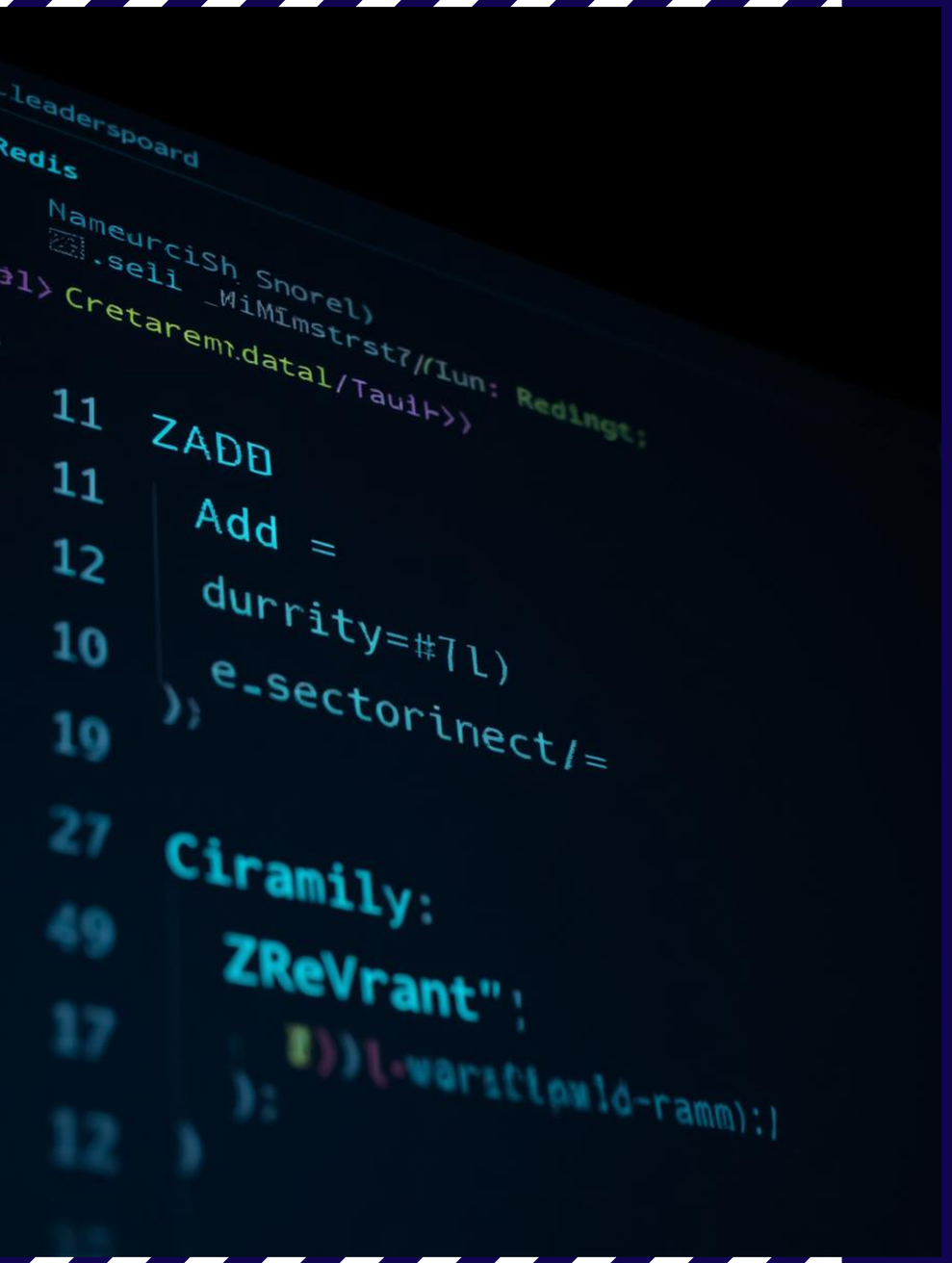


Docker Setup

Starting Redis Container

```
docker run --name redis-leaderboard -p 6379:6379 redis
docker exec -it redis-leaderboard redis-cli
```

- Container = isolated environment
- Docker = container platform



LEADERBOARD DEMO

ZADD leaderboard 1000 mirac / 3000 anthony / 2400 milica

ZREVRANGE leaderboard 0 -1 WITHSCORES

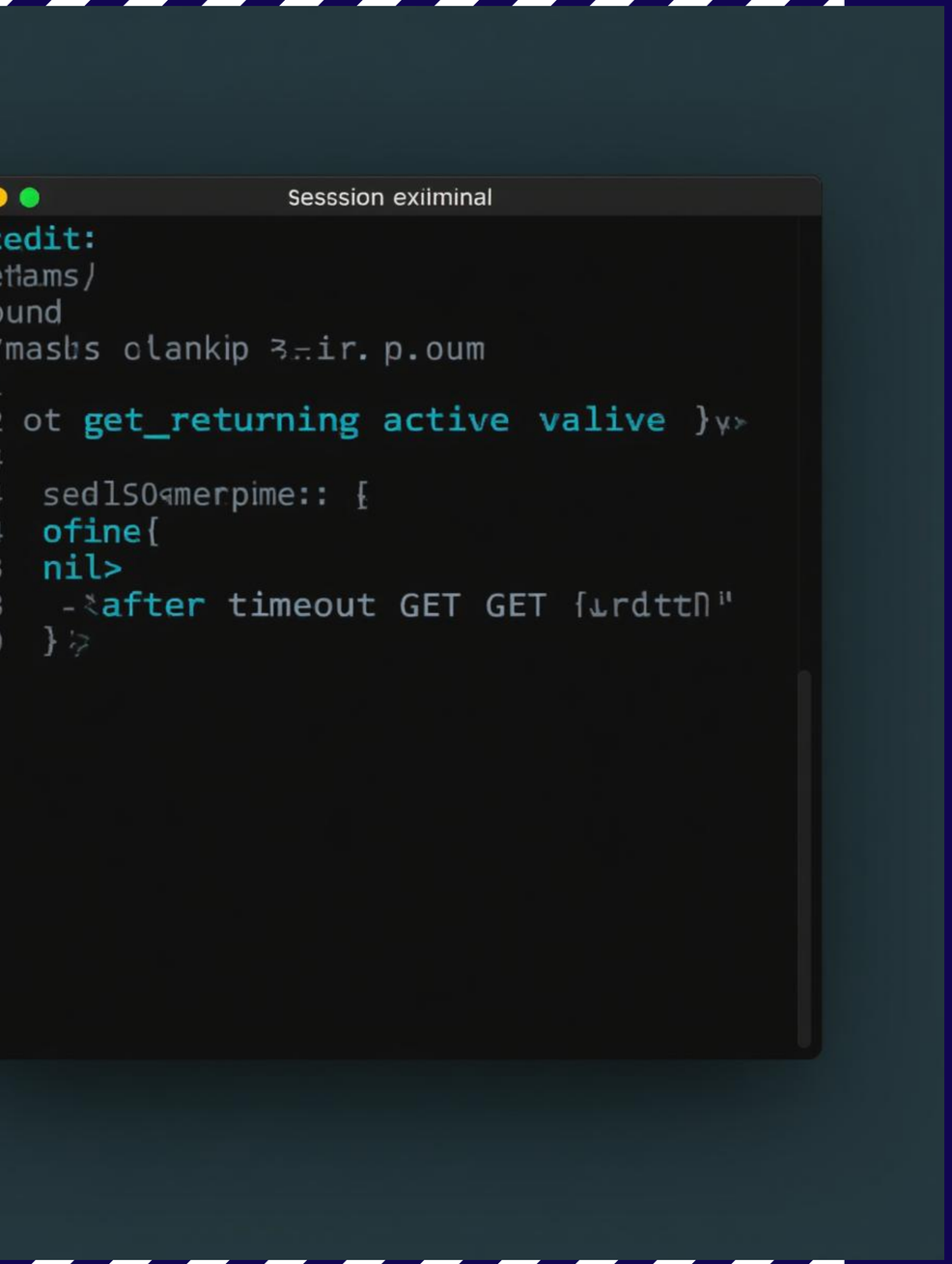
Displays ranked player scores in descending order

UPDATING RANKINGS

ZADD leaderboard 5000 mirac

ZREVRANGE leaderboard 0 -1 WITHSCORES

Redis automatically reorders rankings



SESSION EXPIRATION DEMO

SET session:mirac active EX 10

GET session:mirac → "active"

After 10s: GET → (nil)

- EX = expiration time
- TTL = time to live
- (nil) = key no longer exists

WORKSHOP COMMANDS

```
PING
SET name Mirac
GET name
DEL name

LPUSH tasks study
LRANGE tasks 0 -1

HSET user:1 name Mirac age 20
HGETALL user:1

ZADD leaderboard 1000 player1
ZREVRANGE leaderboard 0 -1 WITHSCORES
```

TUTORIAL OVERVIEW

Docker Setup

Configure Redis container environment, PING/PONG test

Basic key value operations

SET/GET/DEL

Lists

Working with multiple bits of data and how to arrange them

Hashes

Storing multiple fields in a single key and how to call upon them

Sorted Sets

Build ranked data collections

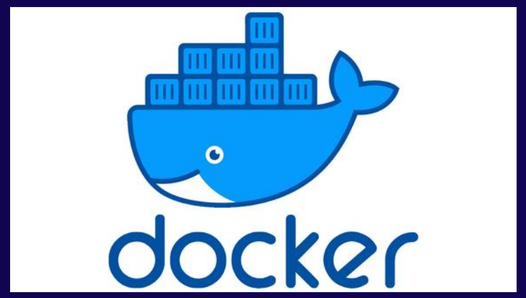
Expiration/sessions

working with EX

Final challenge

Students will make a gaming leaderboard

Part 1 Introduction and setup



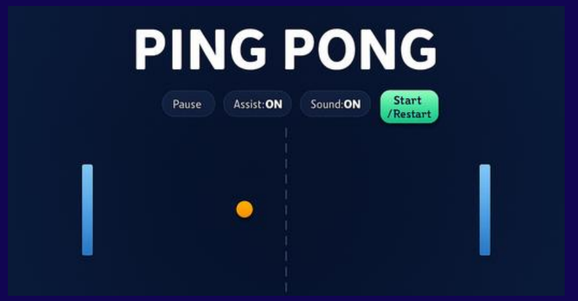
The first thing we shall do is load up our terminal and make sure docker is working.

- Load up your terminal
- type docker--version
- Should output your docker version.

If this doesn't work double check you have docker installed and running

Our next step is to run redis locally on our machines. This saves us from having to install it.

- docker run --name redis-workshop -p 6379:6379 redis
  - **"--Docker run"** tells Docker to create and start a new container, that we will use to run the application instead of installing it
  - **"--name redis-workshop**" gives the container a name
  - **"-p 6379:6379"** maps port 6379 on your machine to port 6379 inside the container. Redis listens on port 6379 by default, so this is what allows your machine to communicate with Redis running inside the container.
  - **"redis"** Docker will look for this on Docker Hub and download it automatically if it isn't already on your machine.
  - We will now open up a new terminal and type in docker exec -it redis-workshop redis-cli



Ping/Pong test

Everyone type PING, if you get a response of Pong congratulations! you have completed part 1.

This is a connectivity test to make sure all our parts are interacting with each other

Part 2 Basic key value operations

This section covers the essential operations of key-value data management: SET, GET, UPDATE, and DELETE. These core actions are vital for efficient data handling, allowing users to store, retrieve, modify, and remove data easily.

When we use SET it creates a key and assigns a value to said key based on the users input.

SET username Mirac

The system then stores this information for later use

When we use GET it searches the systems stored information and retrieves it

GET username

This will then display “Mirac” back to the user

Part 2 Basic key value operations

Update and Delete

When you have to update a value in redis it differs from other tools such as SQL. In SQL you need an “update” command however with redis you can simply just type out the SET command

SET username Anthony

This is because Redis automatically overwrites values without needing that UPDATE command. This is far more efficient

When deleting something you do need the command

DEL username

This removes the key completely. Now when we use a GET operation it will display (nil) which means the key no longer exists

Part 2 Basic key value operations

Student Task

We will now do a short exercise to show you understood the material in part 2

Part 1

- SET name (Enter your first name)
- SET dinner (Enter a food of choice)
- SET drink   (Enter a drink of choice

Now retrieve these values, have the terminal read them back to you

Part 2

- Change dinner to be a different food of your choice
- Delete the drink

Retrieve the values and see if you were successful

Part 3 Lists

A List in Redis is an ordered set of strings, keeping items in the order they were added. One key can hold many values, making Lists useful for things like chat history, queues, waiting rooms, and activity feeds. You can add or remove items from either end, left being referred to as the head and right being referred to as the tail.

LPUSH

Lpush inserts items at the beginning (left/head) of the list. Each new item becomes the new first element

LPUSH queue Mirac
LPUSH queue Anthony
LPUSH queue Milica

Whatever is added last comes first so this would display as
Milica, Anthony, Mirac

LRANGE

To see this we need to use LRANGE as follow

LRANGE queue 0-1

LRANGE works by turning each part of the list into an element starting from 0
0 = entry one
1= entry two
and so on

RPOP

Removes the last/oldest item from the list

RPOP queue

This would remove Mirac as that was the first name added. also making it the oldest.

Part 3 Lists

Student Task

We will now do a short exercise to show you understood the material in part 3

Part 1

- Create a queue with four players
- display the full queue

Part 2

- Now you can delete the oldest one
- Display full queue

Part 4 Hashes

A Hash in Redis stores multiple fields under a single key, similar to a database record or a row in a table. Instead of creating separate keys for each piece of data, you group related attributes together under one key. Hashes are useful for storing user profiles, product information, etc.

This is done using HSET which adds the fields and values to a Hash. A hash is just another way of saying data structure that stores multiple pieces of data under a single key.

This can be done like this

HSET user1: name Anthony age 20 country Scotland

This creates a hash called “user1” which stores our fields and values

Field	Value
NAME	Anthony
AGE	20
COUNTRY	Scotland

Part 4 Hashes

Retrieving our hashes.

There are two ways to retrieve the data in your hash. The first one is HGET which only fetches one single field from the hash. For instance if we use the previous slides example and say

HGET user1 name

I tell the command what hash I am using and what field i want to retrieve this will then return the singular value of

Anthony

You use HGET when you only want to receive one value

Now what if you want to get more than one value back? well you use our second way. HGETALL. the name is quite self explanatory but it returns all the fields and values at once.

HGETALL user1

I again tell the command what hash I am using and it will output all the information related to it

name Anthony age 20 country Scotland

You use HGETALL to display the entire record

Part 4 Hashes

Student Task

We will now do a short exercise to show you understood the material in part 4

Part 1

- Create a Hash called user2
- give this hash a name, an age and a country
- display just the name.

Part 2

- Now with your same hash, display the entire record for it.

Part 5 Sorted sets

A Sorted Set in Redis stores members together with scores. Redis automatically keeps the members sorted based on their scores. Each member is unique and has an associated score that determines its position within the set. If a score changes, Redis automatically updates the ranking order. Saving us from having to do it manually

To create this leaderboard we use ZADD like this

```
ZADD leaderboard 1000  Mirac
ZADD leaderboard 3000 Anthony
ZADD leaderboard 2400 Milica
```

This creates the leaderboard along with adding a score to each member. Redis then looks at each score and automatically sorts the leaderboard based on this.

If we want to then display this data we use

```
ZRANGE leaderboard 0 -1 WITHSCORES
```

This tells the redis, what set we are fetching, how many members we want to display and to include the scores with them. When displayed this will show the set in ascending order

member	score
Mirac	1000
Milica	2400
Anthony	3000

Part 5 Sorted sets

Reverse order and updating

If we wanted our set to be in descending order so the highest score was at the top we would add “REV” to our ZRANGE. REV means reverse.

ZREVRANGE leaderboard 0-1 WITHSCORES

This would now display the leaderboard as follows

member	score
Anthony	3000
Milica	2400
Mirac	1000

Now say Mirac’s score changed and he hit 5000 points, how would we fix this in our set? Its much like our update in part 2 were you don’t need a specific value like “update” you can just allow the redis to overrun it

ZADD leaderboard 5000 Mirac

Now it will automatically sort mirac to the top of the leaderboard so when we run

ZREVRANGE leaderboard 0-1 WITHSCORES

It will display

member	score
Mirac	5000
Anthony	3000
Milica	2400

Part 5 Sorted sets

Student Task

We will now do a short exercise to show you understood the material in part 5

Part 1

- Build your own leaderboard with four players, names and scores
- Display the leaderboard in both ascending and descending

Part 2

- Now update two records scores in the leaderboard and redisplay it to check if redis has automatically sorted it

Part 6 Expiration / Sessions

Redis allows keys to expire automatically after a specified period of time. This is useful when data is only needed temporarily and should be removed automatically. Once the expiration time is reached, Redis deletes the key without requiring any manual intervention.Expiration helps keep data up to date, reduces memory usage, and prevents outdated information from remaining in the database.

First we create a session

SET session mirac EX 20

This creates a session that lasts 10 seconds, if you immediately run

GET session

it calls upon the session and checks if it is still active or not, if it is the output will be “active”

If it has been longer than 10 seconds it will instead output (nill) which means its been removed by redis as the 10 seconds have expired

Real world everday uses such as

- User sessions
- Temporary tokens
- Password reset links
- Cache entries

Student task.

- Create your own session, give it any time you want just nothing too long so you can observe it ending
- Run once during your set time to observe the active output
- Run once off your set time to observe the inactive output (null)

Part 7 Your project

Your final exercise can be chosen between 2 parts that cover different topics and some questions that incorporate all the topics we have covered today.

Challenge 1 — User Management
Build a small user system with Redis

- Save the app name and version of the app (your choice of name and vers)
- Display the app name and version

- Create a login queue of players wanting to join the server
- Display the queue
- delete a player
- redisplay the queue

- Create a user profile
- store all the users information
- display it all
- only display users name

Challenge 2 — Gaming Leaderboard
Build a simple leaderboard that shows player ranks and scores.

- Create a leaderboard with four players and their scores
- Display ranking in ascending and descending

- A player has won a match update a lucky players score of your choice
- Display the rankings again

- Create two active sessions for two players of your choice
- Check the session as you make it
- Check the sessions after a certain amount of time

Questions

- What is the difference between set and get?
- What happens if you use set on a key that already exists?
- What does (nil) mean when Redis returns it?
- What command would you use to delete a key?
- What does Lpush do?
- What does Rpop do?
- What does Lrange queue 0 -1 mean?
- What is the difference between Hget and Hgetall
- Does Redis automatically reorder the rankings?
- What does withscores do?
- What does the ex option do

ANY QUESTIONS?